Chapitre 9 :
BDD et langage SQL

Table des matières

I Introduction **2**

II Modèle relationnnel **2**

III Modèle entité association **4**

IV Langage SQL **7**

V Mots clés SQL hors-programme **13**

I Introduction

Définition

Une **base de donnée** (ou BDD) est un ensemble d'informations organisée de manière à permettre la consultation, modification, sauvegarde, etc… L'accès se fait à travers un Système de Gestion de Base de Données (SGBD)

Pourquoi ?

Une BDD permet en général de résoudre les problèmes suivants :

- Permettre la persistance et la permanence des données, y compris en cas d'extinction imprévue de la machine
- Permettre d'accéder aux données de manière concurrente (plusieurs accès en même temps)
- Garantir l'intégrité et la sécurité des données : quand il arrive une erreur dans une requête de consultation, cela ne doit pas mettre en danger le reste des informations.
- Permettre une mise à jour des données efficace : si on change le prénom de l'auteur d'un livre, on veut ne pas devoir le faire pour tous les livres qui ont été écrits par cet auteur.
- Permettre un accès aux informations à travers une interface standardisée indépendante de la manière dont les données sont vraiment sauvegardées sur le disque

Remarque

Dans la suite on va s'intéresser aux bases de données relationnelles, mais ce n'est pas le seul type de BDD

II Modèle relationnnel

Définition

Ce modèle propose de modéliser les informations d'une base à l'aide d'outils de la logique : **les relations**. Les informations contenues dans la base sont considérées comme des faits qui satisfont des formules logiques. Interroger la base, c'est écrire une formule logique et demander quels sont les faits qui satisfont cette formule.

Définition

Une **relation** est un ensemble de n-uplets, c'est à dire un sous ensemble d'un produit cartésien de n ensembles. Chacun de ces n-uplets est vu comme une valeur pour laquelle la relation est vraie.
On appelle n l'arité de la relation

Les relations sont nommées et leurs différentes dimensions (attribut) le sont aussi. L'ensemble associé à un attribut est appelé son **domaine** (le produit cartésien qui contient la relation est le produit des domaines).

Chaque n-uplet de la relation est un **élément** de cette relation.

Le nombre d'élément d'un relation est son **cardinal**, qui n'est pas l'arité.

Remarque

Les relations sont souvent représentées par des tableaux ou les attributs sont en colonne et les éléments sont les lignes.

Le nombre de colonnes est l'arité et le nombre de lignes est le cardinal.

Exemple 1: Si on prend une relation qui détaille les informations concernant les livres, on peut choisir comme attribut titre, auteur, date de publication.
L'arité de la relation est 3.

Le domaine des titres et des auteurs sont les chaînes de caractères. Le domaine des dates de publication est l'ensemble des entiers.

On appelle sonvent les relations des enregistrements en rapport avec les structures de données des langages de programmation.

Définition

Dans une relation, une **clé primaire** est un ensemble minimal d'attribut de la relation qui permet de représenter de manière unique un élément de la relation.

Une **clé étrangère** est un ensemble d'attributs qui est une clé primaire d'une autre relation

Exemple 2: Dans la relation précédente, {titre, auteur} est une clé primaire. Dans un monde sans homonyme, {auteur}serait une clé étrangère d'une relation regroupant les informations sur les auteurs

🔥 ***Remarque***

Pour éviter et prévenir les problèmes liés aux homonymes ou similaire, il est très courant d'ajouter un attribut supplémentaire qui est un identifiant unique d'un élément de la relation

ℹ **Définition**

On appelle **schéma** d'une relation la donnée du nom de la relation, du nom de chacun de ses attributs et du domaine associé. (analogie avec la signature d'une fonction)

Exemple 3: Livre (titre : string, auteur : string, date de publication : int) est un schéma de la relation précédente.

ℹ **Définition**

Une **base de donnée relationnelle** est un ensemble de relation

Exemple 4: Ici on veut compléter l'exemple précédent pour représenter une base des informations sur les livres. On utilise donc une relation author (id : int, name : string, surname : string, birthdate : int), ainsi qu'une relation wrote (id : int, author_id : int, book_id : int)

Si (i, j, k) est dans wrote, on sait quel'auteur d'identifiant j a écrit le livre d'identifiant k, avec la relation book (id : int, title : string).

Ici, les attributs id sont tous des clés primaires des relations, et author_id et book_id sont des clés étrangères des relations auteur et livre.

III Modèle entité association

Ce modèle permet de représenter de manière synthétique les relations d'une base et leurs liens les uns avec les autres. Mais c'est aussi un moyen de concevoir les structures d'une base de données à partir d'informations en langage naturel.

Des méthodologies précises et documentées pour contruire une base existent mais so,nt hors-programme. Ici on s'intéresse à des concepts de base pour faciliter le processus de construction d'une base et sa représentation.

Il s'agit ici d'une partie de modélisation, une part importante du travail réside dans l'interprétation de l'énoncé et la prise en compte du contexte.

Définition

Le **modèle entité-association** consiste àvoir la base comme un ensemble d'entités liées entre elles par des associations.

- Une **entité** est un objet dont on estime qu'il est pertinent de regrouper les informations ensemble. Une entité est représentées dans un diagramme par un rectangle contenant le nom de l'entité et une liste d'attribut le décrivant.
- Une **association** est un lien entre deux ou plus entités. Ce lien peut être accompagné d'attributs donnant des informations sur ce lin. Ici, on représente les associations par des losanges contenant de le nom de l'association, les attributs éventuels et ce losange est relié aux entités que l'association concerne avec des traits.

Remarque

Il faut distinguer une entité des instances de cette entité. `Livre` est une entité et « Fondation » est une instance de l'entité `Livre`.

Définition

Dans le cas d'une association liant deux entités entre elles, on précise de chaque côté de l'association le nombre maximal d'instance de l'entité qui est concerné par cette association.

Dans le cas d'un nombre d'instance non borné, on note $\star$

On a trois grand types de cardinalités :

- 1-1 (one to one)
  Une instance de la première entité est liée à une listance de la seconde
- 1-$\star$ (one to many)
  Une instance de la première entité est liée à un nombre arbitraire d'instance de la seconde.
- $\star$-$\star$ (many to many)
  Un nombre arbitraire d'instance de la première entité est liée à un nombre arbitraire d'instance de la seconde

(Dans le cas rare où une association concerne plus de deux entités, on peut noter la cardinalité à côté de chaque entité)

🔥 ***Remarque***

Il existe plusieurs standars pour représenter les cardinalités dans un diagramme entité-association. Le programme ne précise aucun standard à utilise, il faut donc précisément systématiquement comment interpréter les cardinalités dans un diagramme

Exemple 5: On considère toujours la bibliothèque

Entités :

- `Livre (titre)`
- `Auteur (nom, prénom, date de naissance)`
- `Maison d'édition (nom)`
- `Utilisateur (nom, prénom)`
- `Adresse (rue, numéro)`

Associations :

- ($\star$-$\star$) `A_écrit` : indique qui a écrit quoi
- (1-$\star$) `En_cours_d'emprunt` ; lie livre et utilisateur
- ($\star$-$\star$) `Publié_par` : lie livre et maison d'édition
- (1-$\star$) `Habite` : lie utilisateur et adresse

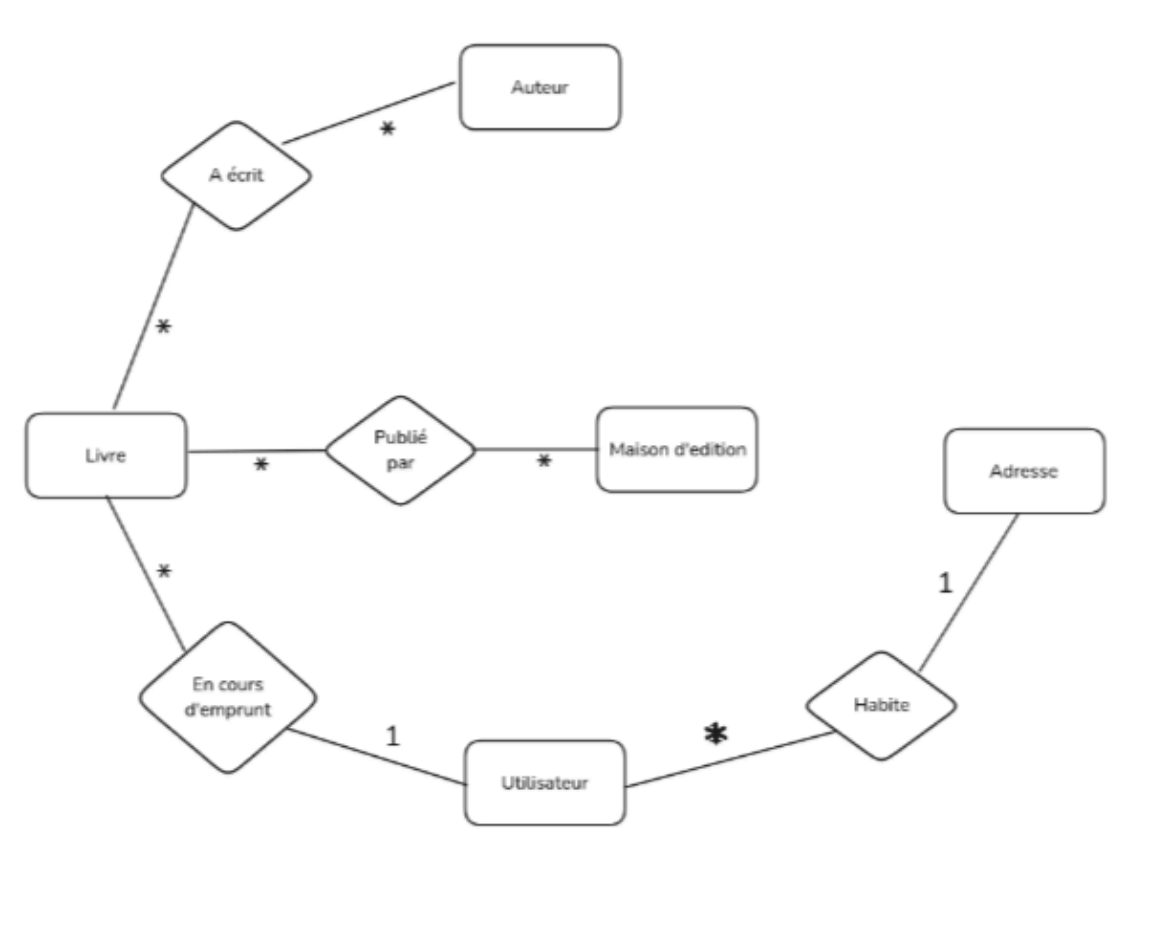

Fig. 1. – Le diagramme entité-association

Conception d'une table à partir d'un diagramme entité-association

- On introduit une relation par entité dont les attributs sont les attributs de l'entité
- Dans le cas d'une cardinalité 1-1, on pourrait fusionner les deux relations, mais aussi ajouter à chaque instance une clé étrangère de l'autre entité
- Pour les 1-$\star$, on rajoute du côté $\star$ une clé étrangère vers l'entité du côté 1.
- Pour les $\star$-$\star$,, on introduit une nouvelle relation représentant cette association dans laquelle on place une clé étrangère de chaque entité associé On parle de réification.

Ici par exemple, on peut obtenir :

Livre : <u>id</u>, titre, *id_emprunteur*
Utilisateur : <u>id</u>, nom, prénom, *id_adresse*
Adresse : <u>id</u>, rue, numéro
Auteur : <u>id</u>, nom, prénom, date_de_naissance
Maison_d'édition : <u>id</u>, nom
Publié_par : <u>id</u>, date, *id_editeur*, *id_livre*
A_écrit : <u>id</u>, *id_auteur*, *id_livre*

IV Langage SQL

Une requète SQL est une formule logique, si on dispose d'une base de donnée dont les relations sont $R_1, ..., R_n$, qui a des valeurs a et b, on peut poser la question : Quels sont les x, y, z tels que $R_{i(x,a,y,b,z)}$ est vraie ?

Exemple 6: Quels sont les prénoms des auteus nés en 1929 se reformule : « Pour quelles valeurs de x la relation `auteur(_, _, x, 1929)` est vraie ? »

🔥 ***Remarque***

Les requêtes SQL décrivent des formules logiques qui sépcifient ce que doit contenir le résultat de la requête, mais ne disent pas comment obtenir le résultat.

Dans le cadre du programme de CPGE, on ne se préoccupera pas de savoir quelles sont les manières efficaces d'obtenir le résultat.

Syntaxe

Par convention, les mots clé de SQL sont en majuscule, ce n'est pas syntaxiquement nécessaire mais c'est universel.

Une requête est constitué de trois éléments principaux :
- La liste des attributs souhaités
- La liste des relations concernés
- Une propriété que vérifient les éléments de la ou les relations

Ce qui donne

```
SELECT <liste des attributs, séparés pae une virgule>
FROM <liste des relations qui interviennent>
WHERE <propriété sur les éléments
```

Exemple 7: Quels sont les auteurs nés en 1929 :

```
SELECT prenom
FROM auteur
WHERE date_de_naissance = 1929
```

La réponse à cette requête est une liste de prénoms, éventuellement vide.

Syntaxe

On peut écrire une propriété à l'aide des opérations logiuqes suivantes :
- `=`, `⩽`, `⩾`, `<`, `>`, `<>`
- `AND`, `OR`, `NOT`

Remarque

Pour lever ules ambiguïtés dans les attributs (éventuellement communs à plusieurs relations), on peut utiliser la syntaxe `relation.attribut`

Exemple 8: Quels sont les titres des livres dont les auteurs sont nés en 1929 ?

```
SELECT L.titre
FROM Livre L, Auteur A, A_ecrit E
WHERE A.date_de_naissance = 1929
      AND L.id = E.id_livre
      AND A.id = E.id_auteur
```

Exemple 9: Quels sont les prénoms des auteurs dont un livre a été publié en 1957 ?

```
SELECT A.prenom
FROM Livre L, Auteur A, A_écrit E, Publié_par P
WHERE P.date = 1951
      AND L.id = P.id_livre
      AND L.id = E.id_livre
      AND A.id = E.id_auteur
```

🔥 ***Remarque***

Si on veut tous les attributs, on peut utiliser * dans le `SELECT`
On peut laisser des commentaires avec – en début de ligne

</> **Syntaxe**

On peut renommer un attribut ou une relation via la syntaxe `<ancien nom> AS <nouveau nom>`.
On peut d'ailleurs ommettre le `AS`, comme fait dans les exemple précédents

Il est possible de réaliser des calculs sur les attributs numériques, les opérateurs autorisés sont : `+, -, *, /, MOD`, on peut réaliser ces calculs dans le `SELECT` ou dans le `WHERE`.

Exemple 10: Quel est l'âge des auteurs nés une année paire ?

```
SELECT (2025 - date_de_naissance) AS age
FROM Auteur
WHERE date_de_naissance MOD 2 = 0
```

</> **Tri et redimensionnement**

Il est possible d'ordonner le contenu d'une requête via un tri selon un critère à l'aide du mot clé `ORDER BY`, placé en fin de requête. Par défaut, il s'agit d'un tri dans l'ordre croissant, mais on peut le spécifier via les mots clés `ASC` pour ascending et `DESC` pour descending.

On peut ne garder que les n premiers éléments d'une requête à l'aide du mot clè `LIMIT n`, placé après un eventuel tri. On peut retirer les k premiers résultats via le mot clé `OFFSET k`. Le mot clè `OFFSET` doit être placé après `LIMIT`

Exemple 11: Quels sont les titres des livres dans l'ordre décroissant des dates de publication, puis croissant des dates de naissance des auteurs ? Ne garder que les résutlats entre 6e et 20e

```
SELECT L.titre
FROM Livre L, Publié_par P, Auteur A, A_écrit E
WHERE L.id = E.id_livre
      AND A.id = E.id_auteur
      AND L.id = P.id_livre
ORDER BY
    P.date DESC,
    A.date_de_naissance ASC
LIMIT 15
OFFSET 5
```

Syntaxe

On peut éliminer les éléments en plusieurs exemplaires dans une requête avec le mot clé `DISTINCT`, placé après `SELECT`, cela concerne le tuple en entier donc on peut avoir des attributs en double tant que les tuples sont distincts

Exemple 12: On souhaite avoir le nom des auteurs sans doublon dont un livre a été publié en 1951

```
SELECT DISTINCT A.prenom A.nom
FROM Auteur A, Livre L, Publié_par P, A_écrit E
WHERE P.date = 1951
      AND P.id_livre = L.id
      AND L.id = E.id_livre
      AND E.id_auteur = A.id
```

Fonctions d'agrégation

Il est possible d'agréger les résultats d'une requête en un seul tuple via les mots clés suivant :

- `COUNT` qui compte le nombre de tuples
- `MIN` et `MAX` qui donne la plus petite et la plus grande valeur
- `SUM` qui fait la somme
- `AVG` qui fait la moyenne

Il est possible de réaliser ces agrégations sur des sous ensembles du résultat via le mot clé `GROUP BY` placé après la requête

`GROUP BY` concerne un tuple d'attributs et tous les éléments de la réponse dont le tuple d'attributs ont la même valeur sont agrégés ensembles. Dans le `GROUP BY`, tout est agrégé ensemble.

Exemple 13: Pour chaque auteur ayant publié un livre, donner le nombre de publications et la date la plus récente

```
SELECT A.nom, COUNT(P.id) AS nb_publi, MAX(P.date) AS plus_récent
FROM Auteur A, Livre L, A_ecrit E, Publié_par P
WHERE A.id = E.id_auteur
    AND L.id = E.id_livre
    AND L.id = P.id_livre
GROUP BY A.id
```

Syntaxe

`WHERE` ne s'applique pas après l'agrégation, one peut donc pas l'utiliser pour avoir par exemple les auteurs à deux publications. À la place, on peut utiliser `HAVING` qui permet de filtrer les résultats après agrégation

Exemple 14: On ne veut garder que les auteurs ayant publié au moins 5 fois

```
SELECT A.nom, COUNT(P.id) AS nb_publi, MAX(P.date) AS plus_récent
FROM Auteur A, Livre L, A_ecrit E, Publié_par P
WHERE A.id = E.id_auteur
    AND L.id = E.id_livre
    AND L.id = P.id_livre
GROUP BY A.id
HAVING COUNT(P.id) ⩾ 5
```

NULL

Il est possible que certains attributs n'aient pas de valeur, dans ce cas on leur donne la valeur `NULL` indépendamment de leur type. Toute comparaison avec `NULL` s'évalue en faux, y compris `NULL = NULL`. On peut détecter cette valeur avec l'opérateur `IS NULL`. Il existe également `IS NOT NULL`. On pourrait substituer `v IS NULL` par `NOT(v=v)` mais on préfère la première option qui est plus lisible.

Remarque

Le résultat d'une requête est une relation, on peut donc faire une requête sur le résultat d'une requête, on parle alors de requête imbriquée.

Exemple 15:

```
SELECT nom
FROM (SELECT *
      FROM auteur
      WHERE prenom = "Ursula")
```

Il faut bien délimiter la requête interne avec des parenthèses, et celà permet de se passer de `HAVING`

Exemple 16:

```
SELECT *
FROM (SELECT A.nom COUNT(P.id) AS nb_publi, MAX(P.date) AS plus_récent
    FROM Auteur A, Livre L, A_ecrit E, Publié_par P
    WHERE A.id = E.id_auteur
          AND L.id = E.id_livre
          AND L.id = P.id_livre
    GROUP BY A.id)
WHERE nb_publi ⩾ 5
```

🔥 ***Remarque***

Si le résultat d'une requête est d'arité 1 et contient un seul élément, on peut l'utiliser comme une valeur dans un calcul ou une comparaison.

🔥 ***Remarque***

Le résultat d'une requête est une relation, une relation est un ensemble de tuples, on peut donc faire des opérations ensemblistes sur le résultat d'une requête.

- `UNION` : réalise l'union des deux requêtes (placé entre les requêtes sans parenthèses)
- `INTERSECT`: réalise l'intersection
- `EXCEPT` (`MINUS` dans certains implémentations) : réalise la différence ensembliste

Cela demande que les relations aient le même schéma (même arité et mêmes noms d'attributs)

</> **Jointure**

On peut réaliser la jointure de deux relations sur la valeur d'un attribut via le mot clé `JOIN ... ON`

Exemple 17:

```
SELECT titre, date
FROM livre
     JOIN edite_par ON livre.id = edite_par.id_livre
```

est équivalent à

```
SELECT livre.titre, edite_par.date
FROM livre, edite_par
```

On peut également enchainer les jointures :

```
SELECT nom, prenom, titre
FROM auteur
JOIN a_ecrit ON auteur.id = a_ecrit.id_auteur
JOIN livre ON livre.id = a_ecrit.id_livre
```

Jointure externe

Il existe un second type de jointure, dit externe, qui permet d'inclure les éléments sans correspondance dans l'autre table. Par exemple dans le cas précédent, on pourrait vouloir inclure des livres non édités

Une jointure classique ne les fera pas apparaître (car il n'existe pas d'élément de édité-par pour lequel id_livre est l'id du livre). Dans une jointure externe, ils apparaissent une fois et les éventuelles valeurs manquantes pour les attributs de la seconde classe sont remplacées par `NULL`

Cette jointure externe n'est pas symétrique, on a donc une jointure gauche et une jointure droite.

Exemple 18: Donner tous les livres et leur date de publication éventuelle. Un seul résultat par livre, avec la date la plus récente

```
SELECT livre.titre, livre.id, MAX(publié_par.date)
FROM livre
     LEFT JOIN publié_par ON livre.id = publié_par.id_livre
GROUP BY livre.id
```

V Mots clés SQL hors-programme

EXISTS

`EXISTS` permet de déterminer si une relation est vide ou non. Si la relation est vide, EXISTS s'évalue en false, et true sinon. Ce mot clé s'applique sur des requetes imbriquées

Exemple 19: Quels sont les auteurs qui n'ont publié aucun livre

```sql
SELECT *
FROM auteur
WHERE NOT EXISTS (SELECT *
                  FROM a_ecrit
                  WHERE a_ecrit.id_auteur = auteur.id)
```

LIKE

Il est possible de vérifier si une chaîne de caractères contient un motif élémentaire via le mot clé `LIKE`. La syntaxe est `<attribut> LIKE <motif>`

Les motifs SQL sont des analogues très restreintes des expressions régulières. On peut utiliser le caractère `%` pour indiquer un nombre quelconque de lettres et `_` pour indiquer une lettre quelconque.

Exemple 20:
`a_` correspond au motif décrivant les chaînes de deux lettres dont la première est un « a ».
`%t` correspond au motif décrivant les chaînes qui terminent par un « t ».
`%et%` correspond aux chaînes de caractères qui contiennent « et »